

Segmentación de Imágenes SEM mediante AutoML: Comparación de Estrategias de Segmentación

Autor

Resumen—Se presenta un enfoque de aprendizaje automático para la detección de zonas frágiles y dúctiles en imágenes obtenidas mediante microscopía electrónica de barrido. El estudio compara modelos del estado del arte basados en transformers con estrategias de segmentación propuestas, integradas en un framework AutoML. La selección del modelo se realiza a partir de métricas de desempeño relevantes, considerando explícitamente las limitaciones computacionales del entorno de ejecución. Los resultados muestran la viabilidad del enfoque bajo recursos restringidos y evidencian el impacto del uso de aumentación de datos y transferencia de aprendizaje en la calidad de la segmentación.

I. INTRODUCCIÓN

La caracterización de los mecanismos de fractura en materiales es un problema central en la ciencia e ingeniería de materiales, debido a su impacto directo en la evaluación del desempeño mecánico y la confiabilidad estructural. En particular, la identificación de zonas asociadas a comportamientos frágiles y dúctiles a partir de microestructuras proporciona información clave para el análisis del fallo y el diseño de materiales.

Las imágenes obtenidas mediante microscopía electrónica de barrido (SEM) constituyen una de las principales herramientas para el estudio de superficies de fractura y microestructuras. No obstante, el análisis tradicional de este tipo de imágenes se basa en inspección visual experta, lo cual introduce subjetividad, limita la reproducibilidad de los resultados y dificulta su aplicación a grandes volúmenes de datos.

En este contexto, las técnicas de aprendizaje automático y, en particular, los modelos de visión por computador han mostrado un potencial significativo para automatizar tareas de clasificación y segmentación en imágenes complejas. Recientemente, modelos basados en transformers, como Vision Transformer (ViT) y Swin Transformer, han alcanzado resultados destacados en diversas tareas de análisis de imágenes, posicionándose como referentes del estado del arte. Sin embargo, su aplicación práctica suele estar condicionada por la disponibilidad de datos anotados y recursos computacionales elevados.

Estas limitaciones motivan la exploración de estrategias alternativas que permitan aprovechar modelos de alto desempeño bajo restricciones de cómputo y datos. En este trabajo se consideran tanto modelos del estado del arte como enfoques propuestos basados en la descomposición espacial de la imagen y el uso de clasificadores preentrenados, con el objetivo de transferir conocimiento desde tareas de clasificación hacia problemas de segmentación.

Adicionalmente, se adopta un enfoque AutoML para automatizar la selección de modelos y el ajuste de hiperparámetros, empleando metaheurísticas y considerando explícitamente las restricciones impuestas por un entorno de cómputo limitado. El objetivo de este estudio es evaluar la viabilidad y el desempeño de estas estrategias para la detección de zonas frágiles y dúctiles en imágenes SEM, comparando modelos del estado del arte con enfoques alternativos bajo condiciones realistas de entrenamiento y evaluación.

II. METODOLOGÍA

III. DESCRIPCIÓN DEL DATASET

IV. ESTRATEGIAS DE SEGMENTACIÓN

IV-A. Vision Transformer (ViT)

IV-B. Swin Transformer

IV-C. Segmentación basada en quadtree

La estrategia de segmentación basada en quadtree adopta un enfoque jerárquico y recursivo, en el cual la imagen se descompone progresivamente en regiones cuadradas de menor tamaño en función de la confianza de un clasificador subyacente. Este método permite reutilizar modelos de clasificación para abordar tareas de segmentación.

El modelo opera sobre imágenes de tamaño fijo y genera máscaras de segmentación a nivel de píxel mediante la clasificación iterativa de regiones. Cada región es evaluada por un clasificador que asigna una etiqueta y una confianza asociada. Si la confianza supera un umbral predefinido, la región se considera homogénea y se asigna directamente a la máscara de salida. En caso contrario, la región se subdivide en cuatro cuadrantes, repitiendo el proceso de forma recursiva.

El criterio de parada de la recursión se define por tres condiciones: alcanzar un nivel mínimo de confianza, llegar a un tamaño mínimo de región o exceder una profundidad máxima de subdivisión. De esta forma, el modelo equilibra precisión espacial y costo computacional, evitando una subdivisión excesiva en regiones con información limitada.

El proceso de entrenamiento se divide conceptualmente en dos etapas. En primer lugar, el clasificador utilizado por el esquema quadtree es entrenado de manera independiente empleando un conjunto de datos de clasificación externo. Este clasificador puede estar basado en redes convolucionales, arquitecturas transformer o cualquier otro modelo, lo que permite transferir conocimiento desde tareas de clasificación hacia la segmentación.

En segundo lugar, el modelo de segmentación ajusta sus hiperparámetros estructurales, en particular el umbral de confianza, el tamaño mínimo de región y la profundidad máxima

del quadtree. Cuando se habilita el modo AutoML, estos hiperparámetros son optimizados mediante una metaheurística de recocido simulado (simulated annealing), utilizando un subconjunto de validación derivado del dataset de segmentación. La optimización se guía por una métrica seleccionada previamente, permitiendo adaptar el comportamiento del quadtree al compromiso deseado entre detalle espacial y generalización.

Una vez finalizado el proceso de optimización, el modelo se configura con la mejor combinación de hiperparámetros encontrada y se evalúa sobre el conjunto completo de entrenamiento.

IV-C1. Clasificador CNN

El clasificador basado en Redes Neuronales Convolucionales (CNN) fue diseñado para clasificar regiones de tamaño variable extraídas por el algoritmo Quadtree.

La arquitectura sigue un patrón de extracción de características seguido de clasificación:

$$\begin{aligned} \text{Entrada} &\rightarrow \text{Bloques Conv.} \rightarrow \text{Pooling Global} \\ &\rightarrow \text{Clasificador} \rightarrow \text{Softmax} \end{aligned} \quad (1)$$

Cada bloque convolucional aplica la siguiente secuencia:

1. Convolución 3×3 con padding=1
2. Batch Normalization
3. Activación ReLU
4. Convolución 3×3 con padding=1
5. Batch Normalization
6. Activación ReLU
7. Max Pooling 2×2 con stride=2

Se usa un kernel de 3×3 porque es el tamaño mínimo que captura información espacial (arriba, abajo, izquierda, derecha y diagonales). Al apilar múltiples capas con kernels pequeños se obtiene un campo receptivo grande con menos parámetros que usando un kernel grande directamente.

El padding de 1 píxel mantiene las dimensiones espaciales constantes después de cada convolución, lo que simplifica el diseño de la red.

Batch Normalization se incluye para acelerar el entrenamiento y estabilizar los gradientes, permitiendo usar tasas de aprendizaje más altas.

Se usan dos convoluciones por bloque antes del pooling para aumentar la capacidad de la red sin reducir las dimensiones espaciales prematuramente.

El parámetro `num_blocks` controla la profundidad de la red. Como cada bloque reduce las dimensiones a la mitad mediante Max Pooling, el tamaño mínimo de entrada es $2^n \times 2^n$ píxeles, donde n es el número de bloques.

Cuadro I
RELACIÓN ENTRE NÚMERO DE BLOQUES Y TAMAÑO MÍNIMO DE ENTRADA

num_blocks	Tamaño mínimo	Canales finales
2	4×4	64
3	8×8	128
4	16×16	256
5	32×32	512

El valor por defecto es 3 bloques, lo que permite clasificar regiones de 8×8 píxeles. Este tamaño fue elegido porque el

Quadtree puede subdividir hasta regiones pequeñas, y 8×8 se consideró suficiente para contener información visual distinguible mientras permite segmentación detallada.

El número de filtros se duplica en cada bloque, comenzando desde 32:

$$\text{filtros en bloque } i = 32 \times 2^{i-1} \quad (2)$$

Esta progresión compensa la reducción de dimensiones espaciales: a medida que la imagen se hace más pequeña, se aumentan los canales para mantener la capacidad de representación.

Antes de la clasificación se aplica `AdaptiveAvgPool2d((1,1))`, que reduce cualquier tamaño espacial a 1×1 calculando el promedio de cada canal.

Esta capa permite que la red acepte entradas de cualquier tamaño (siempre que cumplan el mínimo). Sin importar si la entrada es 8×8 o 256×256, la salida siempre es un vector de tamaño fijo que puede procesarse por las capas densas.

La cabeza de clasificación consiste en:

1. Aplanamiento del tensor
2. Dropout (50 %)
3. Capa densa con reducción a 1/4 de los canales
4. Activación ReLU
5. Dropout (25 %)
6. Capa densa final a 3 clases

El Dropout previene sobreajuste, que es una preocupación con datasets pequeños de imágenes SEM. La capa intermedia reduce la dimensionalidad gradualmente antes de la clasificación final.

La red produce probabilidades mediante Softmax, donde la probabilidad más alta indica la clase predicha y su valor representa la confianza. Esta confianza es usada por el Quadtree para decidir si subdividir la región o aceptar la clasificación.

El entrenamiento usa Cross-Entropy Loss y el optimizador Adam. Las imágenes se agrupan por tamaño para formar batches, ya que las regiones del Quadtree tienen dimensiones variables y solo se pueden apilar en un tensor imágenes del mismo tamaño.

IV-C2. Clasificador ViT

IV-D. Pipeline de Aumentación de Datos

IV-D1. Aumentaciones Geométricas

IV-D2. Aumentaciones Fotométricas

IV-D3. Aumentaciones Específicas para SEM

IV-D4. Composición de Aumentaciones

IV-E. Métricas de Evaluación

IV-E1. Exactitud (Accuracy)

IV-E2. Intersección sobre Unión (IoU)

IV-E3. Coeficiente Dice (F1)

IV-E4. Precisión y Recall

IV-F. Diseño del Framework AutoML

V. RESULTADOS

VI. CONCLUSIONES